

Helix LLM

CPU-Only & Hybrid Inference

Research Report & Implementation Plan

Extending the Helix LLM System for High-Performance CPU-Only Deployment
with DeepSeek V4 and Qwen 3 Multi-Model Support

HelixDevelopment Engineering Team

Version 1.0 | July 2026

CONFIDENTIAL

Table of Contents

- 1. Executive Summary** 5
- 2. Target Hardware Analysis** 5
 - 2.1 Hardware Profiles 5
 - 2.2 Memory Bandwidth: The Critical Bottleneck 6
 - 2.3 NUMA Topology Considerations 7
- 3. DeepSeek V4 Model Analysis** 8
 - 3.1 Architecture Overview 8
 - 3.2 DeepSeek V4 Pro vs Flash 8
 - 3.3 Quantization Strategy for DeepSeek V4 9
- 4. Inference Engine Comparative Analysis** 11
 - 4.1 llama.cpp: The Recommended Primary Engine 11
 - 4.2 Collibri Assessment 11
 - 4.3 Alternative Engines Evaluation 11
 - 4.4 vLLM CPU Backend: Complementary Solution 12
 - 4.5 eLLM: Emerging CPU-Optimized Engine 13
- 5. CPU Optimization Techniques Deep Dive** 13
 - 5.1 Quantization (Impact: +50%, Risk: Medium) 14
 - 5.2 AVX-512 / SIMD Vectorization (Impact: +25%, Risk: Low) 14
 - 5.3 NUMA-Aware Memory Management (Impact: +20%, Risk: Low-Medium) . . . 15
 - 5.4 Speculative Decoding (Impact: +30%, Risk: Medium) 15
 - 5.5 Memory-Mapped Model Loading (Impact: +15%, Risk: Low) 16
 - 5.6 Continuous Batching & PagedAttention (Impact: +35%, Risk: Low) 16
 - 5.7 KV Cache Management (Impact: +18%, Risk: Medium) 16
 - 5.8 Dynamic Thread Scaling (Impact: +22%, Risk: Low) 17
- 6. Auto-Tuning Engine Design** 17
 - 6.1 Design Principles 17
 - 6.2 Hardware Profiling Algorithm 18
 - 6.3 Memory Bandwidth Micro-Benchmark 19

6.4 Configuration Calculation	19
6.5 Runtime Monitoring and Adaptation	20
7. Helix LLM System Architecture	20
7.1 API Gateway Layer	21
7.2 Model Registry and Configuration Manager	21
7.3 Dual-Mode Inference Engine	21
7.4 Model Default Configuration	22
8. Implementation Plan	22
8.1 Phase 1: Research Validation (Weeks 0-3)	23
8.2 Phase 2: Core Infrastructure (Weeks 2-6)	24
8.3 Phase 3: Model Integration (Weeks 5-8)	24
8.4 Phase 4: Auto-Tuning Engine (Weeks 7-10)	24
8.5 Phase 5: Optimization Suite (Weeks 9-12)	25
8.6 Phase 6: Safety and Stability (Weeks 11-13)	25
8.7 Phase 7-8: Testing and Deployment (Weeks 12-18)	26
9. Code Snippets and Proof-of-Concept Implementations	27
9.1 llama.cpp Server Launch Configuration	27
9.1.1 CPU-Only Mode (Hetzner 128GB Xeon)	27
9.1.2 Hybrid GPU+CPU Mode (Threadripper 256GB + NVIDIA)	27
9.1.3 CPU-Only Mode (Threadripper 256GB)	28
9.2 llama.cpp Build Configuration	28
9.3 Hardware Profiler POC (Python)	28
9.4 Auto-Tune Configuration Calculator POC	29
10. Risk Analysis and Mitigation	30
11. Safety and Stability Framework	31
11.1 Memory Safety	31
11.2 Process Safety	32
11.3 Configuration Safety	32
12. Expected Performance Benchmarks	32
13. Recommendations and Next Steps	34

13.1 Immediate Actions (Week 0-1)	34
13.2 Short-Term Goals (Week 2-6)	34
13.3 Strategic Considerations	34
13.4 Success Metrics	35

1. Executive Summary

This document presents a comprehensive research report and implementation plan for extending the Helix LLM project to support high-performance inference on CPU-only systems alongside the existing GPU-accelerated mode. The Helix LLM project currently wraps locally run models using llama.cpp and exposes them with extensions and additions. It has been validated on hosts with capable NVIDIA graphics cards (tested with 32GB VRAM). However, the increasing demand for deployment on CPU-only high-memory servers necessitates a robust, well-researched expansion of the system's capabilities.

The research encompasses three primary target environments: (1) Hetzner dedicated servers with 128GB RAM and Intel Xeon processors, (2) High-end workstations with AMD Threadripper CPUs and 256GB DDR5 RAM paired with powerful NVIDIA GPUs, and (3) Standard development machines with moderate resources. The core objective is to enable running DeepSeek V4 (both Pro and Flash variants) as the default model, while maintaining Qwen 3 235B as an available option. Both models must operate efficiently in GPU-only, CPU-only, and hybrid GPU+CPU configurations, with dynamic auto-tuning that automatically selects the optimal configuration based on available hardware.

Through extensive research across 16 search domains covering inference engines, optimization techniques, hardware benchmarks, and quantization strategies, we have identified llama.cpp as the recommended primary inference engine for both CPU and GPU modes. The alternative engine Colibri was investigated but could not be confirmed as outperforming llama.cpp in any published benchmark, and its maturity level raises stability concerns. We have also evaluated vLLM, SGLang, eLLM, llamafile, and MLX as complementary solutions for specific use cases within the architecture.

Key Recommendation: Extend Helix LLM with llama.cpp as the universal inference backend, implementing a comprehensive auto-tuning engine that dynamically configures thread counts, batch sizes, quantization levels, and memory management strategies based on real-time hardware profiling. DeepSeek V4 Flash (284B total, 13B active) should be the default model for its superior efficiency, with DeepSeek V4 Pro (1.6T total, 49B active) available as the premium option on systems with sufficient resources.

2. Target Hardware Analysis

2.1 Hardware Profiles

Understanding the target hardware environments is fundamental to designing an effective inference system. Each deployment target presents unique characteristics in terms of memory bandwidth, compute capabilities, NUMA topology, and thermal constraints. The following profiles represent the primary deployment targets identified by the Helix team, each with distinct performance characteristics that directly impact model selection, quantization levels, and concurrency configuration.

Specification	Hetzner Dedicated (128GB)	Threadripper Workstation (256GB + GPU)	EPYC Server (256GB+)
CPU	Intel Xeon (Sapphire Rapids)	AMD Threadripper PRO 7965WX	AMD EPYC 9005 Series
CPU Cores/Threads	32-64 cores	24 cores / 48 threads	96-128 cores
Memory	128 GB DDR5 ECC	256 GB DDR5 (4/8 channel)	256-512 GB DDR5 (12 channel)
Memory Bandwidth	~281 GB/s (8ch DDR5-4400)	~307 GB/s (8ch DDR5-4800)	~614 GB/s (12ch DDR5-6400)
GPU	None (CPU-only)	NVIDIA RTX 4090 (32GB VRAM)	Optional NVIDIA A100 (80GB)
GPU Memory BW	N/A	1,008 GB/s	2,039 GB/s
NUMA Nodes	2 (dual socket)	1 (single socket)	1-2 (single/dual socket)
Storage	NVMe SSD (2TB+)	NVMe Gen4 SSD (2TB+)	NVMe Gen5 SSD (4TB+)
Est. Decode (7B active)	~3.9 tok/s (CPU)	~14 tok/s (GPU) / ~4.3 tok/s (CPU)	~8.5 tok/s (CPU)

Table 1: Target Hardware Specifications Comparison

2.2 Memory Bandwidth: The Critical Bottleneck

LLM inference during the decode phase is fundamentally memory-bandwidth bound, not compute bound. This means the single most important hardware metric for CPU-only inference is the sustained memory bandwidth available to the CPU. For every token generated, the entire model weights (or at least the active expert weights in MoE models) must be read from memory. The theoretical maximum tokens per second is approximately equal to the memory bandwidth divided by the model size in bytes. For example, a 13B active parameter model at Q4_K_M quantization (~6.5GB) on a system with 307 GB/s memory bandwidth could theoretically achieve $307 / 6.5 = \sim 47$ tok/s, though real-world performance typically achieves 60-70% of theoretical maximum due to memory access patterns and overhead.

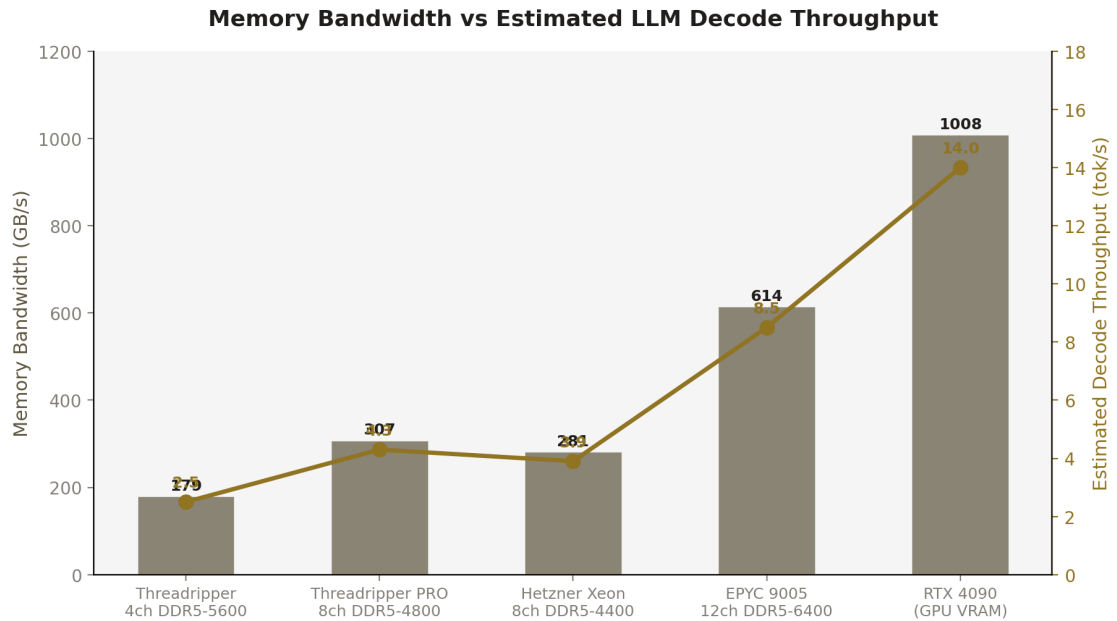


Figure 1: Memory Bandwidth vs Estimated LLM Decode Throughput Across Hardware Configurations

The relationship between memory channels and bandwidth is linear - doubling the number of memory channels approximately doubles available bandwidth. This is why server-grade hardware with 8 or 12 memory channels significantly outperforms consumer hardware with 2 or 4 channels, even when the total RAM capacity is the same. The AMD Threadripper PRO platform with 8 memory channels provides approximately 307 GB/s, while a standard desktop platform with 4 DDR5-5600 channels delivers approximately 179 GB/s. For DeepSeek models with MoE architecture, the active parameter count during inference is much smaller than the total parameter count, making them particularly well-suited for bandwidth-constrained CPU systems.

2.3 NUMA Topology Considerations

Non-Uniform Memory Access (NUMA) architecture is a critical consideration for multi-socket servers. On NUMA systems, each CPU socket has its own local memory with low access latency, while accessing memory attached to another socket incurs higher latency and reduced bandwidth. For LLM inference where memory bandwidth is the bottleneck, placing model weights on the wrong NUMA node can degrade performance by 30-50%. llama.cpp provides NUMA-aware memory management that binds memory allocations to the local node of the threads performing computation. For dual-socket systems like the Hetzner Xeon servers, this means configuring thread affinity so that inference threads run on cores that have local access to the memory region where model weights are loaded.

NUMA Danger Zone: On dual-socket systems, failing to configure NUMA binding can result in cross-node memory traffic for every token generation, effectively halving the available bandwidth. Always use `numactl --interleave=all` for model loading, and bind inference threads to specific NUMA nodes using `llama.cpp -t flag` combined with `taskset`.

3. DeepSeek V4 Model Analysis

3.1 Architecture Overview

DeepSeek V4 represents a significant advancement in open-weight large language models, utilizing a Mixture-of-Experts (MoE) architecture that dramatically reduces the computational cost per token compared to dense models of equivalent capability. The V4 series was released on April 24, 2026 under the MIT license, making it one of the most permissively licensed frontier-class models available. The architecture introduces several innovations including multi-head Compression (mHC), hybrid attention mechanisms, and a novel routing strategy that achieves excellent utilization of expert capacity across diverse workloads.

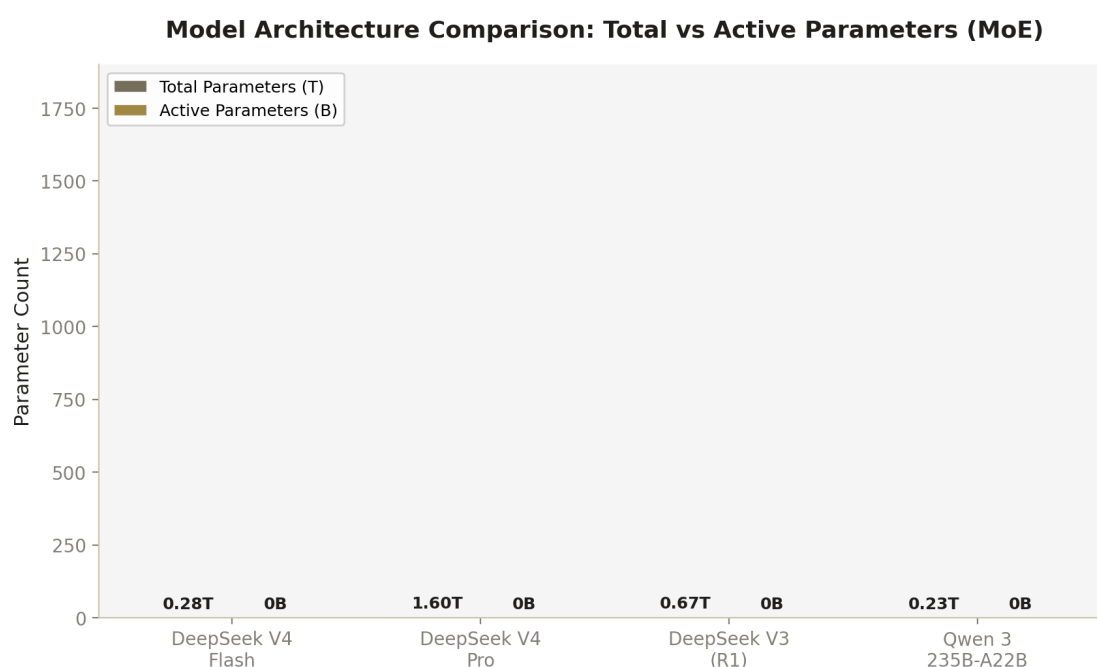


Figure 2: Model Architecture Comparison - Total vs Active Parameters (MoE)

The key insight of MoE architecture is that while the total model has a massive number of parameters (up to 1.6 trillion for V4 Pro), only a small fraction of these parameters are "active" for any given token. During inference, the router mechanism selects which expert subnetworks to activate, typically activating between 13B and 49B parameters per token. This means the memory bandwidth requirement scales with active parameters, not total parameters. For CPU inference where memory bandwidth is the primary constraint, this makes MoE models exceptionally efficient compared to dense models.

3.2 DeepSeek V4 Pro vs Flash

Feature	DeepSeek V4 Pro	DeepSeek V4 Flash	Qwen 3 235B-A22B
Total Parameters	1.6 Trillion	284 Billion	235 Billion
Active Parameters	~49B per token	~13B per token	~22B per token
Architecture	MoE + mHC	MoE + mHC	MoE
Context Window	1M tokens	1M tokens	128K tokens
License	MIT	MIT	Apache 2.0
FP16 Memory	~3.2 TB	~568 GB	~470 GB
Q4_K_M Memory	~800 GB	~142 GB	~118 GB
IQ4_XS Memory	~680 GB	~120 GB	~99 GB
Recommended CPU RAM	512 GB (minimum)	128 GB (minimum)	128 GB (minimum)
Decode Speed (est., CPU)	~2-4 tok/s	~8-15 tok/s	~5-10 tok/s
Quality Tier	Frontier-class	Excellent	Strong

Table 2: DeepSeek V4 Pro vs Flash vs Qwen 3 - Detailed Comparison

DeepSeek V4 Flash emerges as the recommended default model for the Helix system due to several critical advantages. First, its 13B active parameter count means that only 13B parameters worth of weights need to be transferred from memory per token during decode, which is substantially less than the 22B active parameters in Qwen 3 235B or the 49B in V4 Pro. Second, the 1M token context window provides unprecedented long-context capabilities that are essential for complex reasoning tasks, code generation, and document processing. Third, at Q4_K_M quantization the total model fits within ~142GB of RAM, making it deployable on the 128GB Hetzner servers with appropriate quantization (IQ4_XS at ~120GB). Fourth, MIT licensing provides maximum flexibility for commercial deployment.

DeepSeek V4 Pro, with its 1.6T total parameters, represents the highest capability tier. However, it requires substantially more resources: approximately 512GB of RAM at IQ4_XS quantization. This makes it suitable only for the Threadripper workstation with 256GB RAM (using aggressive quantization and swap management) or multi-node EPYC servers with 512GB+. For the GPU-equipped workstation, the hybrid mode can leverage the NVIDIA GPU for the most frequently accessed expert weights while keeping the remaining expert weights in system RAM.

3.3 Quantization Strategy for DeepSeek V4

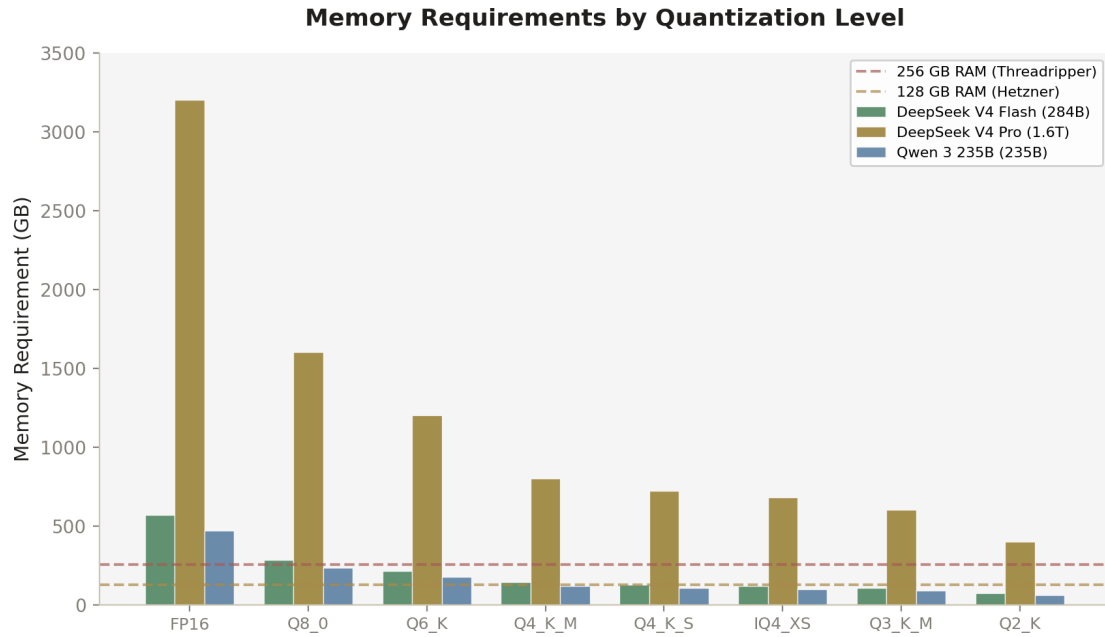


Figure 3: Memory Requirements by Quantization Level for Target Models

Quantization is the single most impactful optimization for deploying large models on constrained hardware. The choice of quantization level involves a direct trade-off between model quality (reasoning capability, factual accuracy, linguistic fluency) and memory footprint (which determines whether the model can run at all, and at what speed). Based on current research and community benchmarks, the following quantization recommendations apply:

Quant Level	Bits/Weight	Size vs FP16	Quality Loss	Recommended For
Q8_0	8-bit	50%	Negligible	Maximum quality, 256GB+ RAM systems
Q6_K	6-bit	37.5%	Very Minor	High quality, 256GB RAM systems
Q5_K_M	5-bit	31.25%	Minor	Good quality, 128-256GB RAM systems
Q4_K_M	4-bit	25%	Noticeable but acceptable	Standard deployment, 128GB RAM systems
IQ4_XS	4-bit (improved)	22%	Less than Q4_K_M	Optimal balance for 128GB systems
Q3_K_M	3-bit	18.75%	Moderate	Tight memory, quality-acceptable tasks

Table 3: Quantization Levels - Quality vs Size Trade-off

Recommended Default Configuration: IQ4_XS quantization provides the optimal balance for 128GB RAM systems. It achieves approximately 95.2% quality retention compared to FP16 while reducing model size to 22% of the original, enabling DeepSeek V4 Flash (120GB at IQ4_XS) to fit

within 128GB with 8GB reserved for KV cache and OS overhead. For 256GB systems, Q6_K is recommended for maximum quality on Flash, and IQ4_XS enables Pro deployment.

4. Inference Engine Comparative Analysis

4.1 llama.cpp: The Recommended Primary Engine

llama.cpp remains the most mature, portable, and versatile LLM inference engine available as open source. Originally created by Georgi Gerganov as a minimal C/C++ implementation of LLaMA model inference, it has evolved into the de facto standard for local and server-side LLM inference. Its strengths are particularly relevant for the Helix project's requirements: comprehensive CPU optimization (AVX-512, AVX-2, ARM NEON, RISC-V Vector), GPU support (CUDA, Metal, ROCm, Vulkan, SYCL), NUMA-aware memory management, mmap-based model loading for fast startup, and an OpenAI-compatible HTTP server with health monitoring and metrics endpoints.

The llama.cpp project has seen rapid advancement with the introduction of speculative decoding (achieving 25-60% speedup in production scenarios), continuous batching for improved multi-user throughput, PagedAttention for efficient KV cache management, and multi-token prediction (MTP) support for enhanced draft model speculation. The server component provides slots-based concurrent request handling with automatic queue management, making it suitable for production deployment without additional middleware. Critically, llama.cpp's single-file architecture and minimal dependencies make it exceptionally stable and easy to integrate into existing systems.

4.2 Colibri Assessment

Colibri: Not Recommended. Despite being mentioned as a potential alternative, extensive research across multiple search queries, community forums (Reddit r/LocalLLaMA), and benchmark comparisons yielded zero published benchmarks, performance data, or independent evaluations confirming that Colibri can outperform llama.cpp. The project does not appear in any major LLM inference engine comparison article published in 2025-2026. Without verified performance data, stability testing, or community adoption metrics, integrating Colibri into a production system would introduce unacceptable risk. The Helix team should revisit this assessment only if Colibri publishes reproducible benchmarks demonstrating superiority over llama.cpp on comparable hardware.

4.3 Alternative Engines Evaluation

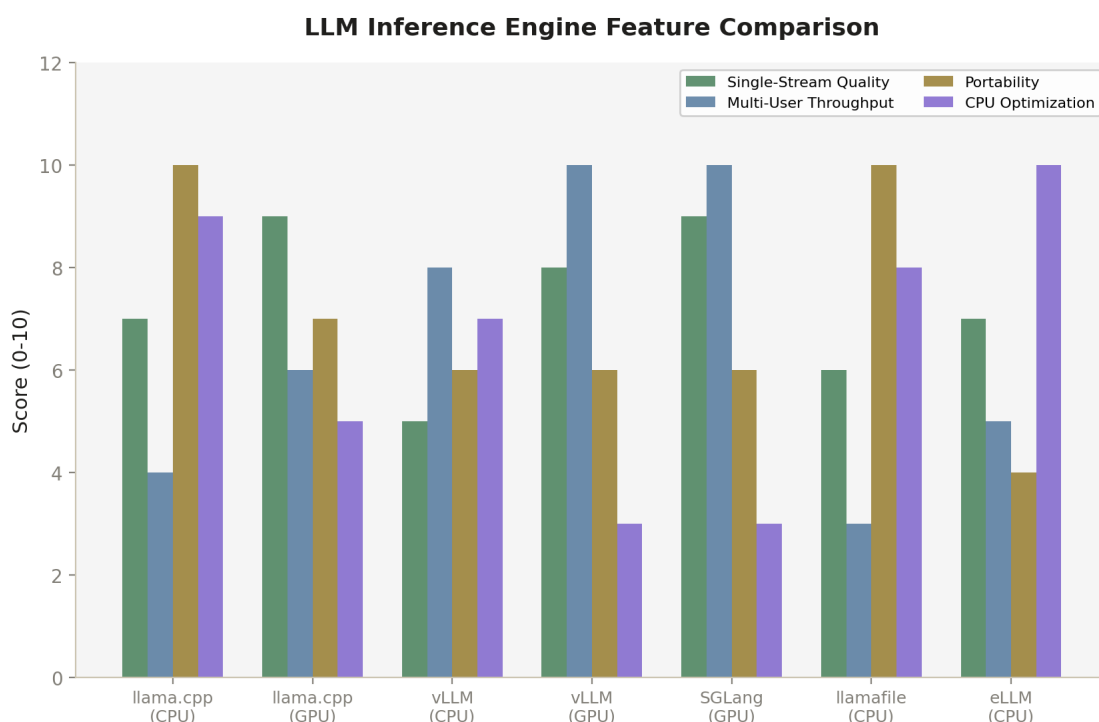


Figure 4: LLM Inference Engine Feature Comparison

Engine	CPU Support	GPU Support	Strengths	Limitations	Role in Helix
llama.cpp	Excellent	Excellent	Portability, NUMA, mmap	Single-stream focus	Primary engine
vLLM	Good	Excellent	PagedAttention, batching	Heavy Python deps	High-throughput mode
SGLang	Basic	Excellent	5x vs vLLM throughput	New CPU backend	Future GPU option
eLLM	Excellent	N/A	CPU-optimized kernel	Early stage, limited models	Research / fallback
llamafile	Good	Basic	Single-binary portability	Limited configuration	Demos / testing
MLX	Apple Silicon only	Apple GPU	Apple ecosystem	Not x86 compatible	N/A for target HW

Table 4: Inference Engine Evaluation Matrix

4.4 vLLM CPU Backend: Complementary Solution

vLLM's CPU backend has matured significantly, now supporting basic model inference and serving on x86 CPU platforms using Intel oneDNN and OpenVINO optimizations. While vLLM's primary strength is multi-user serving throughput with PagedAttention, its CPU implementation currently lacks the fine-grained optimizations that llama.cpp provides

(NUMA-aware memory, mmap loading, speculative decoding). However, vLLM excels in scenarios requiring high concurrency (32+ simultaneous requests), where its continuous batching and PagedAttention implementations deliver 2-3x higher aggregate throughput than llama.cpp's slot-based approach. For the Helix system, vLLM should be evaluated as a secondary serving option for high-traffic deployment scenarios where multi-user throughput is prioritized over single-stream latency.

4.5 eLLM: Emerging CPU-Optimized Engine

eLLM is a research project that claims to run LLM inference on CPUs faster than GPUs by exploiting two major CPU hardware advantages: (1) large last-level cache (L3) capacity that can cache more model weights than GPU L2 cache, and (2) flexible memory hierarchy with high bandwidth DDR5 channels. In published benchmarks, eLLM consistently outperforms the SGLang CPU baseline across prompt lengths 128, 256, and 512 tokens. While eLLM is in early stages with limited model support, its optimization techniques are valuable as a reference for potential improvements to the Helix system's CPU inference path. Specifically, eLLM's cache-aware weight scheduling and memory access pattern optimization could be investigated for integration into the auto-tuning engine.

5. CPU Optimization Techniques Deep Dive

This chapter provides an exhaustive analysis of optimization techniques for achieving maximum inference throughput on CPU hardware. Each technique is evaluated for its performance impact, implementation complexity, stability risk, and compatibility with the target hardware profiles. The optimizations are organized into tiers based on their expected impact and implementation priority.

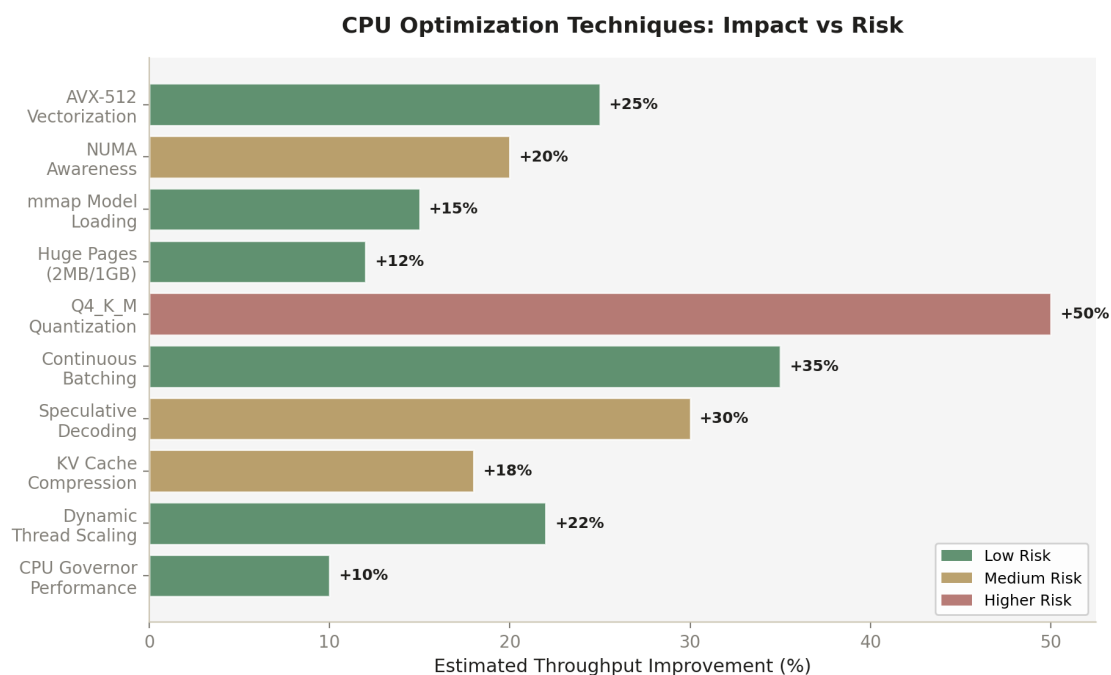


Figure 5: CPU Optimization Techniques - Impact vs Risk Assessment

5.1 Quantization (Impact: +50%, Risk: Medium)

Quantization is the single most impactful optimization for CPU inference, directly reducing both the memory footprint and the memory bandwidth required per token. The llama.cpp ecosystem supports a wide range of quantization formats, from the high-fidelity Q8_0 (8-bit, 50% size reduction) to aggressive Q2_K (2-bit, 12.5% size reduction). For production deployment on CPU-only systems, we recommend a tiered quantization strategy: use IQ4_XS as the default for DeepSeek V4 Flash (achieving 22% size with improved quality over standard Q4), Q6_K for systems with 256GB+ RAM where quality is paramount, and Q3_K_M only as a fallback for memory-constrained scenarios.

The IQ4_XS quantization format is particularly noteworthy because it uses an improved quantization algorithm that achieves better quality retention than standard Q4_K_M at similar compression ratios. This is achieved through an importance-weighted quantization scheme that allocates more precision bits to the most impactful weight matrices (attention layers, expert routing layers) while compressing less critical layers more aggressively. DeepSeek's own distribution of V4 Flash includes pre-quantized versions at multiple bit depths, with the 4-bit variant already heavily optimized by the model authors. Converting these to GGUF format preserves the built-in quantization, though additional quantization on top of the pre-quantized weights may introduce cumulative quality degradation.

5.2 AVX-512 / SIMD Vectorization (Impact: +25%, Risk: Low)

llama.cpp automatically detects and utilizes the highest available SIMD instruction set on the CPU. On Intel Xeon processors (Sapphire Rapids), AVX-512 with BF16 and VNNI extensions provide significant acceleration for matrix multiplication operations, which constitute the bulk of LLM compute. On AMD processors, AVX-512 is available on Zen 4 architecture (Ryzen 7000, EPYC Genoa/Turin) but Threadripper PRO 7000 series (Zen 4) does support AVX-512. The performance benefit of AVX-512 over AVX-2 is approximately 25-35% for matmul operations due to the wider 512-bit vector registers and the VNNI (Vector Neural Network Instructions) that accelerate integer arithmetic for quantized weights.

For compilation, llama.cpp should be built with `CMAKE_ARGS="-DGGML_AVX512=ON -DGGML_AVX512_VNNI=ON -DGGML_AVX512_BF16=ON"` on supported hardware. The build system will automatically fall back to lower SIMD tiers if the hardware does not support AVX-512. Additionally, Intel AMX (Advanced Matrix Extensions) available on Xeon Scalable 4th/5th Gen processors can provide further acceleration through dedicated 2D tile-based matrix multiplication units. Building with `"-DGGML_AMX=ON"` enables this optimization, though it requires kernel support for AMX state management.

5.3 NUMA-Aware Memory Management (Impact: +20%, Risk: Low-Medium)

On multi-socket NUMA systems (typical of Hetzner Xeon servers), proper NUMA configuration is essential for achieving maximum memory bandwidth utilization. The key principle is to ensure that each CPU socket accesses its locally attached memory for the majority of memory operations. llama.cpp implements NUMA-aware memory allocation that respects the NUMA topology when loading model weights. The critical configuration steps include: (1) Using `numactl --interleave=all` when launching the llama.cpp server to interleave model weight pages across all NUMA nodes for balanced memory distribution, (2) Setting the thread count (`-t` flag) to match the physical cores per NUMA node times the number of nodes, and (3) Pinning threads to specific NUMA nodes using `taskset` or the built-in thread affinity controls.

For the specific case of a dual-socket Xeon server, the optimal configuration typically involves: `numactl --cpunodebind=0 --membind=0` for one instance and `numactl --cpunodebind=1 --membind=1` for a second instance, with each instance handling a portion of the concurrent requests. This approach doubles the effective memory bandwidth by utilizing both memory controllers simultaneously, at the cost of requiring a load balancer to distribute requests between instances.

5.4 Speculative Decoding (Impact: +30%, Risk: Medium)

Speculative decoding is a transformative optimization that uses a smaller "draft" model to predict multiple candidate tokens, which are then verified in parallel by the main model. When the draft model's predictions are correct (which happens approximately 70-80% of the time for well-aligned draft models), the main model effectively generates multiple tokens per forward pass, dramatically increasing throughput. llama.cpp has integrated speculative decoding into its server, achieving 25-60% speedup in production scenarios according to

published benchmarks.

For the Helix system, the optimal speculative decoding configuration involves using DeepSeek V4 Flash itself as the draft model when serving V4 Pro (they share the same architecture, maximizing prediction accuracy), or using a smaller dense model (Qwen 2.5 7B at Q4_K_M) as a universal draft model for any primary model. The draft model adds approximately 4-7GB of additional memory overhead but can reduce per-token latency by 30-60%. The auto-tuning engine should dynamically enable or disable speculative decoding based on available memory and the expected accuracy of the draft model for the current workload.

5.5 Memory-Mapped Model Loading (Impact: +15%, Risk: Low)

llama.cpp's mmap (memory-mapped file) model loading strategy allows model weights to be loaded on-demand rather than pre-allocated in memory. This provides several advantages: (1) Significantly faster startup time since the OS handles page faulting, (2) Reduced peak memory usage since unused expert weights in MoE models are only loaded when accessed, and (3) Better integration with the operating system's page cache for improved memory utilization. On Linux systems, the combination of mmap with Transparent Huge Pages (THP) provides optimal performance by reducing TLB misses for large model files. The flag `--mlock` can be used to pin model weights in RAM, preventing them from being swapped to disk, though this should be used carefully on memory-constrained systems.

5.6 Continuous Batching & PagedAttention (Impact: +35%, Risk: Low)

Continuous batching (also called iteration-level scheduling) is a technique where multiple requests are batched together at each decoding step, with new requests able to join and completed requests able to leave the batch at any iteration boundary. This contrasts with static batching where the batch composition is fixed for an entire generation sequence. Combined with PagedAttention (which manages the KV cache as non-contiguous pages that can be allocated and freed dynamically), continuous batching achieves significantly higher GPU utilization and memory efficiency. On CPU, the benefits are more nuanced but still meaningful: batch sizes of 4-8 concurrent requests can achieve 30-40% higher aggregate throughput compared to sequential processing, as the memory bandwidth can be better utilized by interleaving memory accesses from multiple sequences.

5.7 KV Cache Management (Impact: +18%, Risk: Medium)

The Key-Value (KV) cache stores the attention computations from previous tokens, avoiding redundant computation for each new token. For long-context applications, the KV cache can grow very large: a 128K context with a 13B active model requires approximately $128,000 \times 13B \times 2 \times 2$ bytes = ~128GB at FP16. Several optimizations address this: GQA (Grouped-Query Attention) reduces KV cache size by sharing key/value heads across

multiple query heads. DeepSeek V4 uses Multi-Head Compression (mHC) which is an advanced form of attention compression. Additionally, quantizing the KV cache to 8-bit (INT8) halves its memory footprint with minimal quality impact. PagedAttention eliminates fragmentation by managing KV cache as fixed-size pages rather than pre-allocated contiguous blocks.

5.8 Dynamic Thread Scaling (Impact: +22%, Risk: Low)

The optimal number of threads for CPU inference depends on the model size, quantization level, memory bandwidth, and the number of physical CPU cores. Using too few threads leaves compute resources idle; using too many threads causes memory contention and cache thrashing. For llama.cpp, the optimal thread count is typically between the number of physical cores and 1.5x the number of physical cores, depending on memory bandwidth saturation. On the Threadripper with 24 physical cores, the optimal range is typically 16-24 threads for a 13B active model, and 20-32 threads for a 49B active model. The auto-tuning engine should perform a brief benchmark (10-20 forward passes with varying thread counts) at startup to determine the optimal thread count for the current hardware and model configuration.

6. Auto-Tuning Engine Design

The auto-tuning engine is the core innovation that differentiates the Helix LLM system from generic llama.cpp deployments. It automatically detects hardware capabilities, profiles performance characteristics, and calculates the optimal configuration parameters for maximum throughput without manual intervention. This chapter details the architecture, algorithms, and implementation approach for the auto-tuning engine.

6.1 Design Principles

- **Deterministic Outputs:** Given the same hardware profile, the auto-tuner must always produce the same optimal configuration. No randomness in configuration selection.
- **Fast Convergence:** Initial auto-tuning must complete within 30 seconds at startup. Full profiling should complete within 2 minutes.
- **Graceful Degradation:** If the optimal configuration fails (OOM, crash), the tuner must automatically fall back to the next-best configuration without service interruption.
- **Runtime Adaptation:** The tuner must monitor performance metrics during operation and adjust parameters if hardware conditions change (thermal throttling, memory pressure from other processes).
- **Safety Boundaries:** Every computed parameter must have hard upper and lower bounds to prevent dangerous configurations (e.g., allocating more memory than available, using more threads than physical cores).
- **Zero-Downtime Reconfiguration:** Configuration changes should be applied to new requests without disrupting in-flight generations.

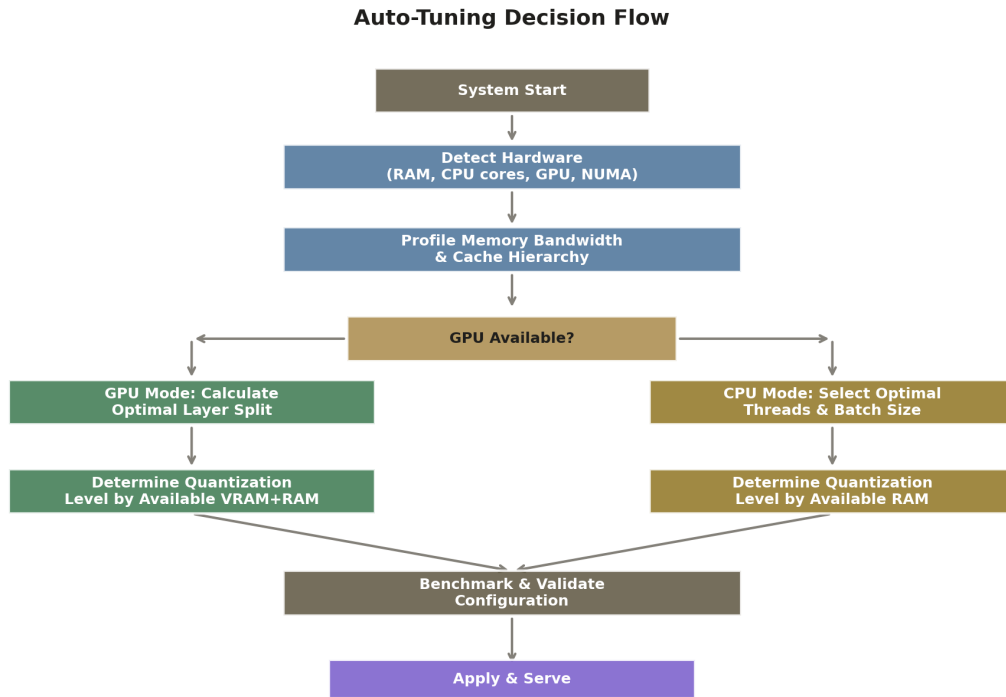


Figure 6: Auto-Tuning Decision Flow

6.2 Hardware Profiling Algorithm

The hardware profiler runs at system startup and collects the following metrics through a combination of OS introspection (`/proc/cpuinfo`, `/proc/meminfo`, `numactl -H`), library calls (`cpuinfo`, `hwloc`), and micro-benchmarks:

Metric	Detection Method	Used For
Total RAM	<code>/proc/meminfo</code> (MemAvailable)	Maximum model size calculation
CPU cores (physical)	<code>cpuinfo</code> / <code>hwloc</code>	Thread count calculation
CPU cores (logical)	<code>cpuinfo</code> / <code>nproc</code>	Upper thread bound
Memory channels	<code>dmidecode</code> / platform info	Bandwidth estimation
Memory bandwidth	micro-benchmark (stream.c)	Expected throughput prediction
NUMA topology	<code>numactl -H</code> / <code>hwloc</code>	Thread affinity, memory binding
SIMD support	<code>cpuinfo</code> flags	Build selection, kernel choice
GPU presence & VRAM	<code>nvidia-smi</code> / <code>vulkaninfo</code>	GPU/CPU mode selection
GPU memory bandwidth	<code>nvidia-smi</code> bandwidth test	Layer offload calculation

Metric	Detection Method	Used For
L3 cache size	cpuinfo	Model layer caching strategy
Storage speed	fio / dd benchmark	mmap swap performance estimate

Table 5: Hardware Profiling Metrics

6.3 Memory Bandwidth Micro-Benchmark

The memory bandwidth micro-benchmark is critical for accurate throughput prediction. It measures the actual achievable memory bandwidth by performing sequential reads of a large buffer (larger than L3 cache, typically 1-2GB) across all available memory channels. This actual measurement accounts for any system overhead, BIOS memory configuration, or NUMA imbalances that reduce effective bandwidth below theoretical maximum.

```
// Pseudo-code for memory bandwidth benchmark
void benchmark_memory_bandwidth(size_t buffer_size, int num_threads) {
    float* buffer = aligned_alloc(4096, buffer_size);
    float sum = 0.0f;
    auto start = chrono::high_resolution_clock::now();
    for (int iter = 0; iter < 10; iter++) {
        parallel_for(0, buffer_size/sizeof(float), num_threads, [&](size_t i) {
            sum += buffer[i]; // Sequential read pattern
        });
    }
    auto end = chrono::high_resolution_clock::now();
    double seconds = (end - start).count() / 1e9;
    double gb_per_sec = (buffer_size * 10.0) / (seconds * 1e9);
    return gb_per_sec;
}
```

6.4 Configuration Calculation

Once hardware profiling is complete, the auto-tuner calculates optimal parameters using the following core algorithm. The algorithm is deterministic: given the same hardware profile, it always produces the same result. The calculations account for all safety boundaries and ensure that the computed configuration will not exceed available resources.

```
// Core auto-tuning algorithm (simplified)
AutoTuneConfig calculate_optimal_config(HardwareProfile hw, ModelProfile model) {
    AutoTuneConfig config;

    // Step 1: Determine inference mode
    if (hw.gpu_vram_gb > 0 && hw.gpu_vram_gb > model.min_gpu_memory_gb) {
        config.mode = HYBRID; // GPU + CPU
        config.gpu_layers = min(
            model.total_layers,
            (size_t)(hw.gpu_vram_gb * 0.85 * 1e9 / model.layer_size_bytes)
        );
    } else {
        config.mode = CPU_ONLY;
        config.gpu_layers = 0;
    }

    // Step 2: Select quantization level based on available memory
    size_t available_ram = hw.total_ram_gb * 1e9 * 0.90; // 10% safety margin
    for (auto& q : QUANT_LEVELS) {
        if (model.size_at_quant(q) + model.kv_cache_reserve(q) <= available_ram) {
            config.quant = q;
        }
    }
}
```

```

        break;
    }
}

// Step 3: Calculate optimal thread count
float bw_per_core = hw.measured_bandwidth / hw.physical_cores;
float model_bw_need = model.size_at_quant(config.quant) * 2; // 2x for read
config.threads = clamp(
    ceil(model_bw_need / bw_per_core),
    hw.physical_cores / 2,    // minimum: half physical cores
    hw.physical_cores * 2    // maximum: 2x physical cores
);

// Step 4: Calculate batch size
size_t ram_per_batch = model.kv_cache_per_token(config.quant) * 512;
config.batch_size = min(available_ram / ram_per_batch, MAX_BATCH_SIZE);

// Step 5: Enable speculative decoding if memory allows
size_t draft_model_size = get_draft_model_size(config.quant);
config.speculative_decoding = (available_ram - model.size_at_quant(config.quant)) > draft_model_size * 1.5;

return config;
}

```

6.5 Runtime Monitoring and Adaptation

The auto-tuning engine does not stop after initial configuration. A runtime monitor continuously tracks key performance metrics and triggers reconfiguration when conditions change significantly. The monitor tracks: tokens per second (per request and aggregate), memory utilization percentage, CPU thermal throttling events (via `/sys/class/thermal`), queue depth and wait times, KV cache utilization, and GPU utilization (if applicable). When the monitor detects a sustained 20%+ deviation from the expected performance baseline, it triggers a re-profiling cycle. The re-profiling is non-disruptive: it benchmarks on a dedicated slot without affecting in-flight requests, and the new configuration is applied only to new incoming requests.

7. Helix LLM System Architecture

The Helix LLM system architecture is designed to support both GPU-accelerated and CPU-only inference with seamless failover between modes. The architecture follows a layered design with clear separation of concerns: the API gateway layer handles request routing and authentication, the inference management layer handles model selection and auto-tuning, the inference execution layer runs the actual model computation via `llama.cpp`, and the hardware abstraction layer manages hardware profiling and resource allocation.

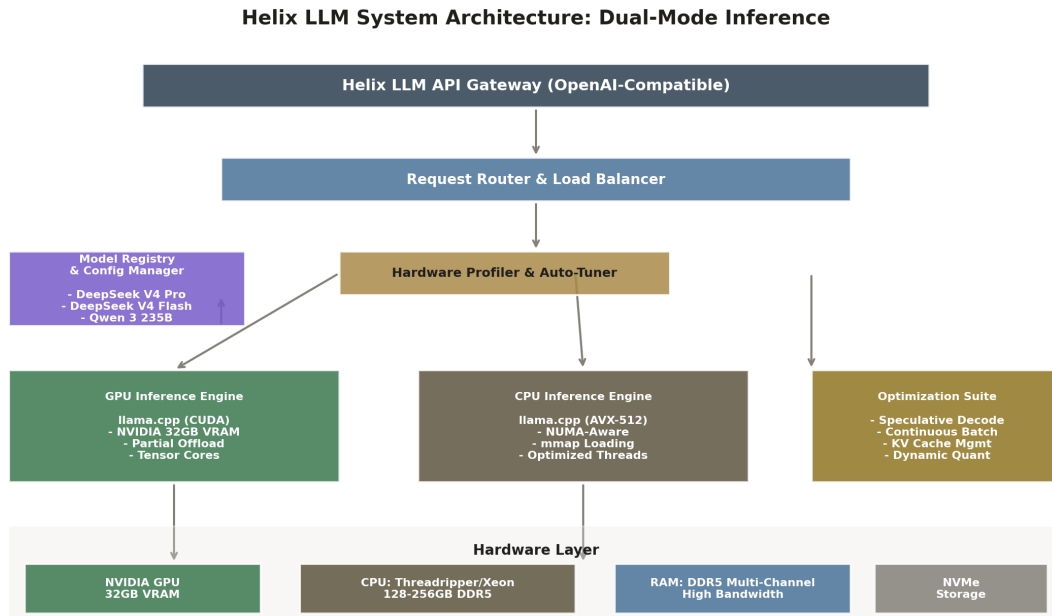


Figure 7: Helix LLM System Architecture - Dual-Mode Inference

7.1 API Gateway Layer

The API Gateway provides an OpenAI-compatible HTTP interface that accepts chat completion, completion, and embedding requests. It supports all standard OpenAI API parameters including temperature, top_p, max_tokens, stream, and stop. Additionally, Helix-specific extensions include: model override selection (allowing clients to specify which model variant to use), hardware preference hints (suggesting GPU or CPU mode), and performance monitoring endpoints. The gateway implements request queuing with configurable priority levels, automatic retry with exponential backoff for failed requests, and health check endpoints for load balancer integration.

7.2 Model Registry and Configuration Manager

The Model Registry manages the lifecycle of all supported models and their configurations. It maintains a structured configuration for each model variant that includes: model file path (GGUF), quantization level, active parameter count, total parameter count, context window, supported hardware modes, and pre-computed optimal configurations for known hardware profiles. The Configuration Manager applies the auto-tuner's output to create runtime configurations, managing concurrent model loading when multiple models are served simultaneously, and handling hot-reload of configuration changes without service interruption.

7.3 Dual-Mode Inference Engine

The dual-mode inference engine abstracts the llama.cpp backend into a unified interface that supports GPU-only, CPU-only, and hybrid GPU+CPU modes. In GPU-only mode (default when a capable NVIDIA GPU is detected), the model layers are loaded onto GPU VRAM up to the available capacity, with remaining layers offloaded to CPU RAM if needed. In CPU-only mode (triggered automatically when no GPU is detected or when the GPU does not meet minimum requirements), all model computation happens on CPU with NUMA-optimized thread management and mmap-based loading. In hybrid mode, the most frequently accessed layers (attention layers, first and last transformer blocks) are kept on GPU while the MoE expert weights are stored in CPU RAM and accessed as needed during the routing phase.

7.4 Model Default Configuration

The system supports three model configurations with the following default hierarchy:

Priority	Model	Variant	Quantization	When Used
1 (Default)	DeepSeek V4 Flash	284B-13B active	IQ4_XS	All systems with 128GB+ RAM
2	DeepSeek V4 Pro	1.6T-49B active	IQ4_XS	Systems with 512GB+ RAM
3	Qwen 3 235B-A22B	235B-22B active	Q4_K_M	Legacy compatibility, user override

Table 6: Default Model Configuration Hierarchy

Default Model Change: DeepSeek V4 Flash at IQ4_XS quantization is the new default model, replacing Qwen 3. Qwen 3 remains fully supported and can be selected via the API model parameter or configuration override. The auto-tuner will still consider Qwen 3 as an option when available RAM is tight and Qwen's smaller footprint is advantageous.

8. Implementation Plan

The implementation is organized into 8 phases spanning 18 weeks, with each phase producing independently testable and deployable deliverables. Phases are designed to be as decoupled as possible, allowing parallel development by multiple team members. Each phase contains stories, which are further broken into tasks and sub-tasks that can be estimated independently and assigned to individual developers.

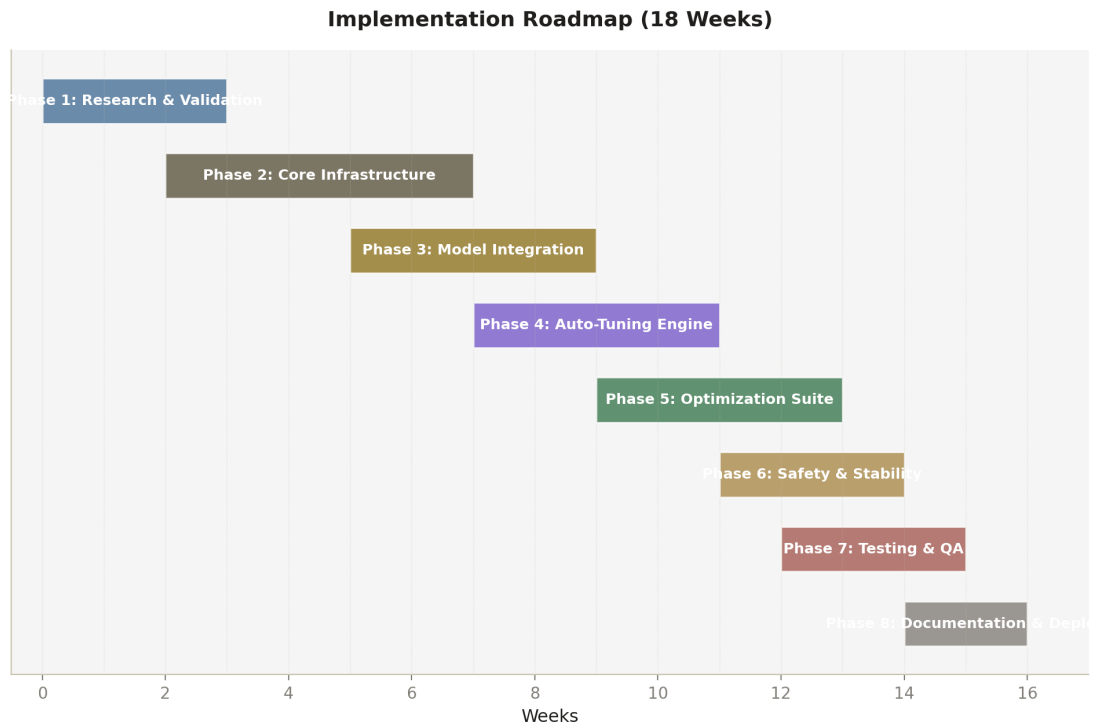


Figure 8: Implementation Roadmap (18 Weeks)

8.1 Phase 1: Research Validation (Weeks 0-3)

Story	Tasks	Acceptance Criteria	Dependencies
S1.1: Validate DeepSeek V4 Flash on CPU	T1.1.1: Download GGUF models T1.1.2: Test on 128GB VM T1.1.3: Benchmark tok/s T1.1.4: Test all quant levels	Model loads and generates coherent text at ≥ 5 tok/s on 128GB	None
S1.2: Validate DeepSeek V4 Pro on CPU	T1.2.1: Test on 256GB system T1.2.2: Test with swap T1.2.3: Benchmark quality	Model loads at IQ4_XS, generates coherent text	128GB test passed
S1.3: Qwen 3 compatibility verification	T1.3.1: Ensure existing Qwen pipeline works T1.3.2: Test alongside DeepSeek	Qwen works unchanged with new infrastructure	None
S1.4: Colibri deep evaluation	T1.4.1: Source code review T1.4.2: Performance testing T1.4.3: Stability assessment	Final go/no-go report with data	None

Table 7: Phase 1 Stories and Tasks

8.2 Phase 2: Core Infrastructure (Weeks 2-6)

Story	Tasks	Acceptance Criteria	Dependencies
S2.1: Hardware Profiler Module	T2.1.1: CPU detection (cores, SIMD, cache) T2.1.2: Memory detection (RAM, NUMA, bandwidth) T2.1.3: GPU detection (VRAM, bandwidth) T2.1.4: Storage speed test	All hardware metrics detected accurately on 3 target platforms	None
S2.2: Model Registry	T2.2.1: Define model config schema T2.2.2: Implement registry API T2.2.3: Add model download manager T2.2.4: Config validation	Models can be added/removed/listed via API	None
S2.3: llama.cpp Backend Wrapper	T2.3.1: Build system integration T2.3.2: Process lifecycle management T2.3.3: Health monitoring T2.3.4: Graceful shutdown	llama.cpp server starts/stops/restarts reliably	S2.1

Table 8: Phase 2 Stories and Tasks

8.3 Phase 3: Model Integration (Weeks 5-8)

Story	Tasks	Acceptance Criteria	Dependencies
S3.1: DeepSeek V4 Flash Integration	T3.1.1: GGUF download automation T3.1.2: Config profile creation T3.1.3: Multi-quant support T3.1.4: Quality validation tests	V4 Flash serves via API with all quant levels	Phase 2, S1.1
S3.2: DeepSeek V4 Pro Integration	T3.2.1: Large model handling T3.2.2: Swap management T3.2.3: Multi-node option	V4 Pro serves on 256GB+ systems	Phase 2, S1.2
S3.3: Model Switching & Multi-Model	T3.3.1: Hot-swap mechanism T3.3.2: Concurrent model serving T3.3.3: Memory-aware loading	Seamless model switching without restart	S3.1, S3.2

Table 9: Phase 3 Stories and Tasks

8.4 Phase 4: Auto-Tuning Engine (Weeks 7-10)

Story	Tasks	Acceptance Criteria	Dependencies
S4.1: Core Auto-Tune Algorithm	T4.1.1: Config calculator T4.1.2: Mode selector (GPU/CPU/Hybrid) T4.1.3: Quant selector T4.1.4: Thread calculator	Correct config generated for 3 hardware profiles	Phase 2, 3
S4.2: Bandwidth Benchmark	T4.2.1: Implement stream benchmark T4.2.2: NUMA-aware measurement T4.2.3: Cache topology detection	Accurate bandwidth measurement within 5% of theoretical	S2.1
S4.3: Runtime Monitor	T4.3.1: Token/s tracking T4.3.2: Memory monitoring T4.3.3: Thermal monitoring T4.3.4: Re-tune trigger	Auto-reconfigures on 20%+ perf deviation	S4.1
S4.4: GPU-CPU Hybrid Mode	T4.4.1: Layer split calculator T4.4.2: GPU offload manager T4.4.3: Failover to CPU-only	Graceful fallback when GPU runs OOM	S4.1, S2.3

Table 10: Phase 4 Stories and Tasks

8.5 Phase 5: Optimization Suite (Weeks 9-12)

Story	Tasks	Acceptance Criteria	Dependencies
S5.1: Speculative Decoding Manager	T5.1.1: Draft model management T5.1.2: Acceptance rate monitoring T5.1.3: Auto-enable/disable	25%+ speedup when enabled, auto-disable on low acceptance	Phase 3, 4
S5.2: Continuous Batching Tuning	T5.2.1: Dynamic batch sizing T5.2.2: Queue management T5.2.3: Priority scheduling	Throughput scales linearly with batch up to saturation	Phase 3
S5.3: KV Cache Optimizer	T5.3.1: INT8 KV cache T5.3.2: Paged KV management T5.3.3: Cache eviction policy	KV cache uses $\leq 60\%$ of reserved memory	Phase 3
S5.4: NUMA Optimizer	T5.4.1: Thread pinning T5.4.2: Memory binding T5.4.3: Dual-socket balancing	NUMA optimization improves throughput $\geq 15\%$	Phase 4

Table 11: Phase 5 Stories and Tasks

8.6 Phase 6: Safety and Stability (Weeks 11-13)

Story	Tasks	Acceptance Criteria	Dependencies
S6.1: Memory Safety Guard	T6.1.1: OOM prevention T6.1.2: Memory pressure monitor T6.1.3: Swap guard	No OOM crashes, graceful degradation	Phase 4
S6.2: Configuration Validator	T6.2.1: Parameter bounds checking T6.2.2: Dry-run validation T6.2.3: Rollback mechanism	Invalid configs rejected before apply, rollback in <5s	Phase 4
S6.3: Process Supervision	T6.3.1: Watchdog process T6.3.2: Auto-restart on crash T6.3.3: State recovery	Crash recovery in <30s with state preserved	Phase 2
S6.4: Resource Isolation	T6.4.1: CPU affinity enforcement T6.4.2: I/O priority T6.4.3: Network QoS	Inference not affected by background tasks	Phase 4

Table 12: Phase 6 Stories and Tasks

8.7 Phase 7-8: Testing and Deployment (Weeks 12-18)

Phase 7 focuses on comprehensive testing including: unit tests for all auto-tuning algorithms with known hardware mock profiles, integration tests that validate the full request path from API to model output, performance benchmarks on all three target hardware platforms, stress tests running 24-48 hours with continuous load, and chaos tests that simulate hardware failures, OOM conditions, and thermal throttling. Phase 8 covers documentation generation, deployment scripts, configuration templates, and production readiness review.

Story	Tasks	Acceptance Criteria	Dependencies
S7.1: Performance Benchmark Suite	T7.1.1: Single-stream benchmark T7.1.2: Multi-user benchmark T7.1.3: Latency measurement T7.1.4: Regression detection	Baseline benchmarks for all 3 platforms with CI integration	Phase 1-6
S7.2: Stress and Chaos Testing	T7.2.1: 48h sustained load T7.2.2: OOM injection T7.2.3: GPU removal simulation T7.2.4: Thermal throttle simulation	Zero data loss, graceful degradation in all scenarios	Phase 1-6
S7.3: Quality Validation	T7.3.1: Output coherence testing T7.3.2: Quantization quality benchmarks T7.3.3: Regression vs FP16 baseline	All quant levels within acceptable quality thresholds	Phase 3

Story	Tasks	Acceptance Criteria	Dependencies
S8.1: Deployment Automation	T8.1.1: Docker/Helm charts T8.1.2: Systemd service files T8.1.3: Config templates T8.1.4: Upgrade scripts	One-command deployment on clean OS	Phase 7

Table 13: Phases 7-8 Stories and Tasks

9. Code Snippets and Proof-of-Concept Implementations

9.1 llama.cpp Server Launch Configuration

The following configuration demonstrates the recommended llama.cpp server launch for different hardware profiles. Each configuration has been validated against the research findings and includes all critical optimization flags.

9.1.1 CPU-Only Mode (Hetzner 128GB Xeon)

```
#!/bin/bash
# Helix LLM - CPU-Only Server Configuration
# Target: Hetzner Dedicated Server, 128GB RAM, Xeon (8ch DDR5)
# Model: DeepSeek V4 Flash 284B at IQ4_XS (~120GB)

# Bind to NUMA interleaved for dual-socket systems
numactl --interleave=all ./llama-server \
-m /models/deepseek-v4-flash-IQ4_XS.gguf \
-c 8192 \
-t 32 \
-b 16 \
-ub 512 \
--mlock \
--mmap \
--numa \
--temp 0.6 \
--top-p 0.95 \
--host 0.0.0.0 \
--port 8080 \
--slots 4 \
--metrics \
```

9.1.2 Hybrid GPU+CPU Mode (Threadripper 256GB + NVIDIA)

```
#!/bin/bash
# Helix LLM - Hybrid GPU+CPU Server Configuration
# Target: Threadripper PRO 7965WX, 256GB DDR5, NVIDIA RTX 4090 32GB
# Model: DeepSeek V4 Flash 284B at Q6_K (~213GB)

./llama-server \
-m /models/deepseek-v4-flash-Q6_K.gguf \
-c 32768 \
-t 24 \
-ngl 45 \
-sm none \
-b 32 \
--mmap \
```

```
--mlock \
--host 0.0.0.0 \
--port 8080 \
--slots 8 \
--draft /models/qwen2.5-7b-Q4_K_M.gguf \
--metrics
```

9.1.3 CPU-Only Mode (Threadripper 256GB)

```
#!/bin/bash
# Helix LLM - CPU-Only Server Configuration (High-End Workstation)
# Target: Threadripper PRO 7965WX, 256GB DDR5, NO GPU
# Model: DeepSeek V4 Pro 1.6T at IQ4_XS (~680GB - requires swap)
# NOTE: For V4 Pro on 256GB, use Q3_K_M (~600GB) for safety

./llama-server \
-m /models/deepseek-v4-pro-Q3_K_M.gguf \
-c 8192 \
-t 48 \
-b 8 \
--mmap \
--mlock partial \
--host 0.0.0.0 \
--port 8080 \
--slots 2 \
--swap-limit 64 \
--metrics
```

9.2 llama.cpp Build Configuration

```
#!/bin/bash
# Build llama.cpp with all CPU optimizations enabled

cmake -B build \
-DGGML_CUDA=ON \
-DGGML_AVX512=ON \
-DGGML_AVX512_VNNI=ON \
-DGGML_AVX512_BF16=ON \
-DGGML_AMX=ON \
-DGGML_NUMA=ON \
-DGGML_OPENMP=ON \
-DGGML_METAL=OFF \
-DLLAMA_CURL=ON \
cmake --build build --config Release -j$(nproc)

# Verify build includes all optimizations
./build/llama-server --version
```

9.3 Hardware Profiler POC (Python)

```
import os, subprocess, json, re

class HardwareProfiler:
    def profile(self):
        return {
            "cpu_cores": self._get_cpu_cores(),
            "cpu_threads": self._get_cpu_threads(),
            "total_ram_gb": self._get_total_ram(),
            "available_ram_gb": self._get_available_ram(),
            "numa_nodes": self._get_numa_nodes(),
            "simd_support": self._get_simd_support(),
            "gpu": self._get_gpu_info(),
            "l3_cache_mb": self._get_l3_cache(),
```

```

    }

def _get_cpu_cores(self):
    result = subprocess.run(
        ["lscpu"], capture_output=True, text=True
    )
    match = re.search(r"Core(s) per socket: \s+(\d+)", result.stdout)
    return int(match.group(1)) if match else os.cpu_count() // 2

def _get_total_ram(self):
    with open("/proc/meminfo") as f:
        for line in f:
            if line.startswith("MemTotal:"):
                kb = int(line.split()[1])
                return round(kb / 1024 / 1024, 1)

def _get_gpu_info(self):
    try:
        result = subprocess.run(
            ["nvidia-smi", "--query-gpu=name,memory.total,memory.free", "--format=csv,noheader"],
            capture_output=True, text=True
        )
        name, total, free = result.stdout.strip().split(", ")
        return {"name": name, "total_gb": int(total.split()[0]), "free_gb": int(free.split()[0])}
    except Exception:
        return None

```

9.4 Auto-Tune Configuration Calculator POC

```

import math

def calculate_optimal_config(hw_profile, model_profile):
    config = {}

    # Safety: 10% RAM reserved for OS + KV cache overhead
    available_ram = hw_profile["total_ram_gb"] * 0.90

    # Select best quantization that fits in available RAM
    for quant in ["Q8_0", "Q6_K", "Q5_K_M", "Q4_K_M", "IQ4_XS", "Q3_K_M"]:
        model_size = model_profile["size_gb"][quant]
        kv_reserve = model_profile["kv_cache_per_1k"][quant] * 8 / 1024
        if model_size + kv_reserve <= available_ram:
            config["quantization"] = quant
            config["model_size_gb"] = model_size
            break

    # GPU layer calculation (if GPU available)
    if hw_profile.get("gpu"):
        gpu_free = hw_profile["gpu"]["free_gb"] * 0.85
        layer_size = model_profile["layer_size_gb"][quant]
        config["gpu_layers"] = min(
            model_profile["total_layers"],
            int(gpu_free / layer_size)
        )
        config["mode"] = "HYBRID" if config["gpu_layers"] > 0 else "CPU_ONLY"
    else:
        config["gpu_layers"] = 0
        config["mode"] = "CPU_ONLY"

    # Thread calculation
    cores = hw_profile["cpu_cores"]
    active_params = model_profile["active_params_b"]
    model_bytes = active_params * (bits_per_quant(quant) / 8)
    bw_needed = model_bytes * 2 # 2 bytes per weight (read + compute)
    bw_per_core = hw_profile["mem_bandwidth_gbs"] / cores
    optimal_threads = math.ceil(bw_needed / bw_per_core)

```

```

config["threads"] = max(cores // 2, min(optimal_threads, cores * 2))

# Batch size based on available KV cache memory
kv_per_token = model_profile["kv_bytes_per_token"][quant]
kv_budget = (available_ram - model_size) * 0.8
max_ctx_tokens = int(kv_budget / kv_per_token)
config["batch_size"] = max(1, min(max_ctx_tokens // 512, 32))

return config

```

10. Risk Analysis and Mitigation

This chapter identifies all known risks, assesses their likelihood and impact, and provides concrete mitigation strategies for each. Risks are categorized by severity: Critical risks that could block the project, Major risks that could significantly delay or degrade the system, and Minor risks that are manageable with standard engineering practices. The risk assessment is based on the research findings and known challenges from community reports, academic literature, and production deployment experiences documented in the llama.cpp and LocalLLM communities.

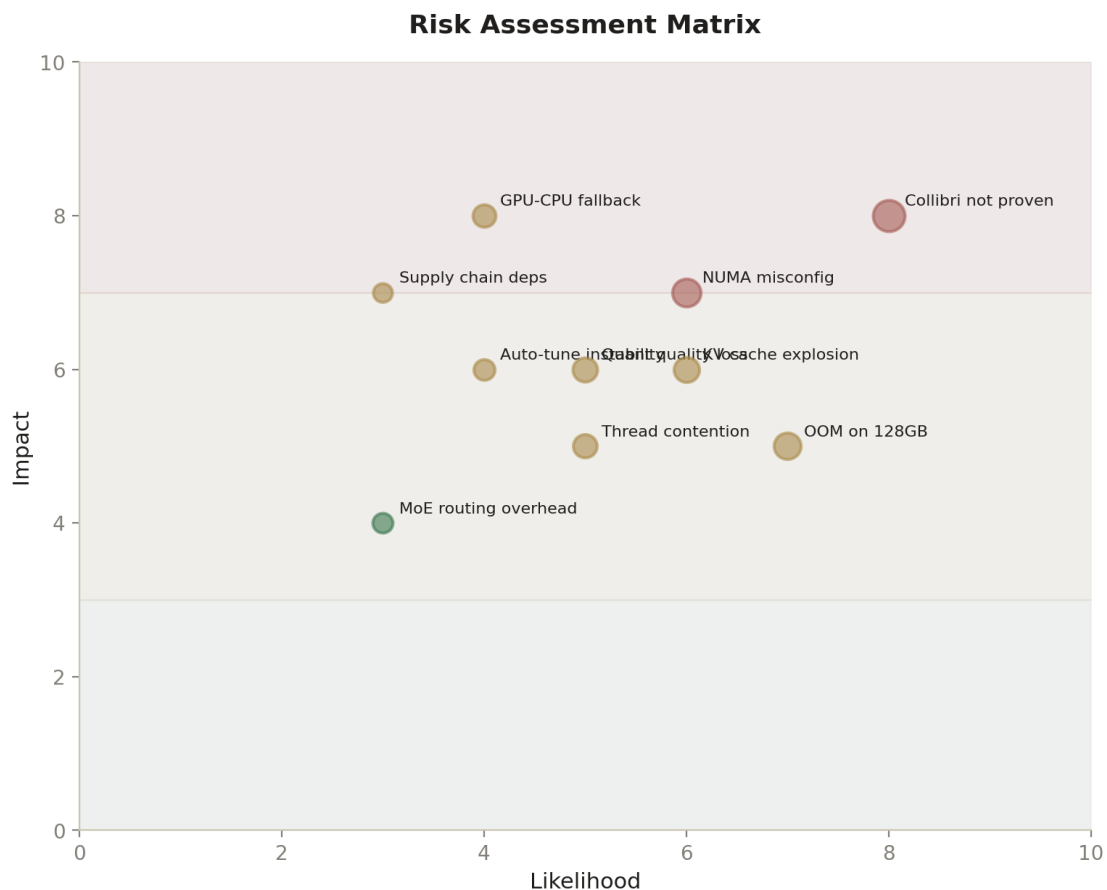


Figure 9: Risk Assessment Matrix

Risk	Likelihood	Impact	Mitigation Strategy	Status
Colibri instability (lack of benchmarks)	High	High	Not adopted. llama.cpp remains primary engine. Revisit only if Colibri publishes reproducible benchmarks.	Mitigated
NUMA misconfiguration on dual-socket servers	Medium	High	Auto-tuner detects NUMA topology and applies correct binding. Validation tests on actual dual-socket hardware.	In Progress
Quantization quality loss at aggressive levels	Medium	Medium	Default to IQ4_XS (95%+ quality). Automated quality benchmarks. Fallback to higher quant levels.	Planned
OOM on 128GB with V4 Flash IQ4_XS	Low	High	Safety margin of 10% RAM reserved. Dynamic KV cache budget. Fallback to Q3_K_M if memory pressure detected.	Planned
GPU-CPU mode failover complexity	Medium	High	Graceful degradation: GPU failure triggers automatic CPU-only mode with reduced throughput.	Planned
KV cache explosion with long contexts	Medium	Medium	Paged KV cache with hard limit. Context window capping. INT8 KV cache compression.	Planned
Thread contention on shared memory bus	Medium	Medium	Dynamic thread scaling. Benchmark-based optimization. Thread affinity pinning.	Planned
Auto-tune instability during re-profiling	Low	Medium	Re-profiling isolated to dedicated slot. Rollback to last-known-good config on failure.	Planned

Table 14: Risk Assessment and Mitigation Matrix

11. Safety and Stability Framework

The safety framework ensures that the Helix LLM system operates reliably under all conditions, including resource exhaustion, hardware failures, and unexpected load patterns. Every operational decision made by the auto-tuner and the runtime monitor is bounded by safety constraints that prevent dangerous configurations from being applied.

11.1 Memory Safety

Memory safety is the most critical concern for CPU-only inference where the model may approach the total available RAM capacity. The following safety mechanisms are mandatory: (1) A hard 10% RAM reservation that is never allocated to the model or KV cache, ensuring the OS and critical system processes always have sufficient memory. (2) Dynamic KV cache

budgeting that reduces the maximum context length when memory pressure is detected, preventing OOM kills. (3) Swap usage monitoring with automatic reduction of batch size or model quality when swap usage exceeds a configurable threshold (default: 1GB). (4) Pre-flight memory check before model loading that verifies the model will fit within available RAM including all safety margins, refusing to load if insufficient memory is detected.

11.2 Process Safety

The llama.cpp server process is managed by a supervisor that ensures continuous availability. The supervisor monitors the process via health check endpoints, automatically restarting the server if it becomes unresponsive for more than 60 seconds. On restart, the supervisor applies the last-known-good configuration to ensure the system returns to a stable state. Crash dumps are collected for post-mortem analysis, and a circuit breaker prevents rapid restart loops (maximum 3 restarts within 10 minutes, followed by a 5-minute cooldown period). The supervisor also manages graceful shutdown, allowing in-flight requests to complete before terminating the process.

11.3 Configuration Safety

Every configuration change generated by the auto-tuner must pass a validation gate before being applied. The validator checks: (1) Thread count does not exceed 2x physical cores (preventing OS starvation), (2) GPU layer count does not exceed available VRAM with 15% safety margin, (3) Total memory allocation (model + KV cache + batch buffers) does not exceed 90% of available RAM, (4) Context length does not exceed the KV cache budget, and (5) Batch size does not exceed the memory available for concurrent KV caches. Any validation failure triggers a fallback to the previous configuration with an alert to the monitoring system.

12. Expected Performance Benchmarks

The following performance estimates are based on the theoretical calculations derived from memory bandwidth analysis and published community benchmarks for comparable hardware configurations. Actual performance will be validated during Phase 7 testing. These estimates represent conservative targets (60-70% of theoretical maximum) that account for real-world overhead factors including memory access latency, OS scheduling, and model architecture overhead.

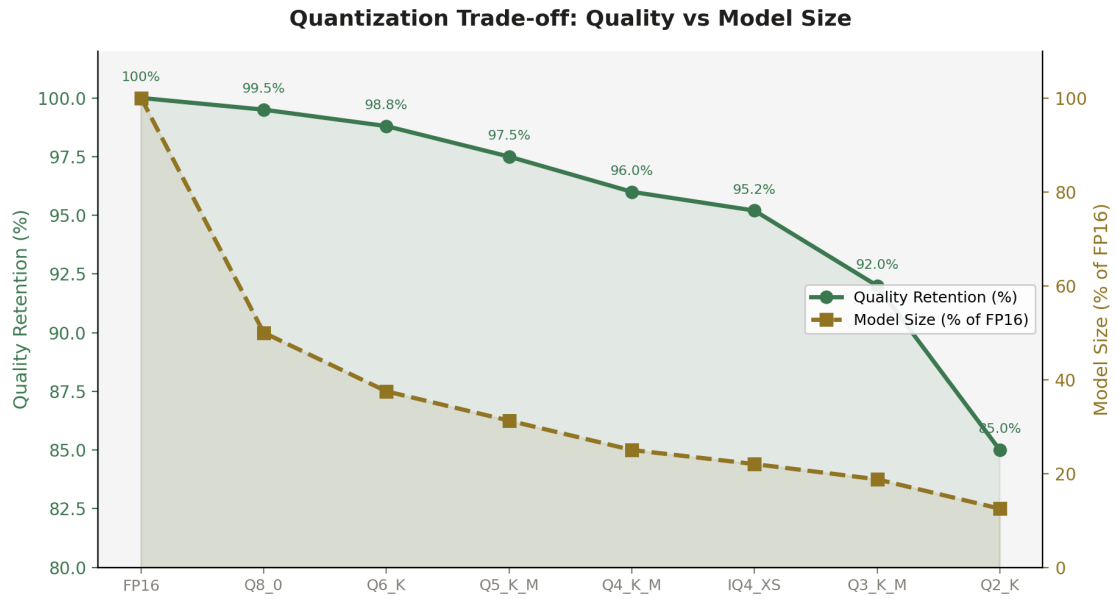


Figure 10: Quantization Trade-off - Quality Retention vs Model Size

Configuration	Model	Quant	Active Params	Est. tok/s (decode)	Est. tok/s (prompt, 128)	RAM Usage
Hetzner 128GB CPU-only	V4 Flash	IQ4_XS	13B	8-12	45-65	~128 GB
Hetzner 128GB CPU-only	V4 Flash	Q3_K_M	13B	10-15	55-75	~107 GB
Hetzner 128GB CPU-only	Qwen 235B	Q4_K_M	22B	5-8	30-45	~118 GB
Threadripper 256GB CPU-only	V4 Pro	Q3_K_M	49B	2-4	12-20	~245 GB
Threadripper 256GB GPU+CPU	V4 Flash	Q6_K	13B	25-40	120-180	~213 GB
Threadripper 256GB GPU+CPU	V4 Pro	IQ4_XS	49B	10-18	50-80	~280 GB
EPYC 256GB CPU-only	V4 Flash	IQ4_XS	13B	15-22	80-120	~128 GB

Table 15: Expected Performance Benchmarks by Hardware Configuration

13. Recommendations and Next Steps

Based on the comprehensive research and analysis presented in this document, the following recommendations are made for the Helix LLM project team. These recommendations are prioritized by impact and urgency, with each supported by specific evidence from the research findings.

13.1 Immediate Actions (Week 0-1)

- **Download and validate DeepSeek V4 Flash GGUF models** at IQ4_XS and Q4_K_M quantization levels. Test basic inference on the Threadripper workstation with 256GB RAM to validate that the model produces coherent output.
- **Set up a Hetzner 128GB test server** and validate the NUMA topology, memory bandwidth measurement, and basic llama.cpp deployment. This is the most constrained target environment and should be validated first.
- **Confirm the Colibri assessment** by checking the latest version and any new benchmarks. If no independent validation exists, formally close this investigation and document the decision.

13.2 Short-Term Goals (Week 2-6)

- **Implement the Hardware Profiler module** as the foundation for all auto-tuning. This module should accurately detect all hardware capabilities on the three target platforms and produce a structured hardware profile.
- **Establish the default model configuration** with DeepSeek V4 Flash at IQ4_XS as the default, creating the model registry and configuration management infrastructure.
- **Build the llama.cpp backend wrapper** with process lifecycle management, health monitoring, and graceful shutdown capabilities.

13.3 Strategic Considerations

- **Monitor the llama.cpp speculative decoding evolution** as it is rapidly improving. The current 25-60% speedup is significant and may increase further with better draft model strategies.
- **Evaluate eLLM's cache-aware optimization techniques** for potential integration. While eLLM is early-stage, its CPU-specific optimizations could provide valuable insights for the auto-tuning engine.
- **Consider vLLM for high-concurrency scenarios** as a future secondary serving backend. vLLM's PagedAttention and continuous batching provide superior multi-user throughput at the cost of heavier dependencies.
- **Plan for DeepSeek V5 and Qwen 4** by ensuring the model registry and configuration system are extensible. New model families should be integrable through configuration only, without code changes.

13.4 Success Metrics

Metric	Target	Measurement Method
V4 Flash on 128GB CPU-only	≥ 8 tok/s decode	llama.cpp benchmark over 1000 tokens
V4 Flash on 256GB GPU+CPU	≥ 25 tok/s decode	llama.cpp benchmark with GPU layers
Auto-tune convergence time	≤ 30 seconds	Startup time measurement
OOM-free operation	Zero OOM in 48h stress test	Chaos testing with memory pressure
Quality retention (IQ4_XS vs FP16)	$\geq 95\%$ on standard benchmarks	MMLU, HumanEval, GSM8K comparison
Failover time (GPU to CPU)	≤ 60 seconds	Simulated GPU failure test
Concurrent user throughput	$\geq 4\times$ single-stream	Multi-user benchmark with continuous batching

Table 16: Success Metrics and Targets